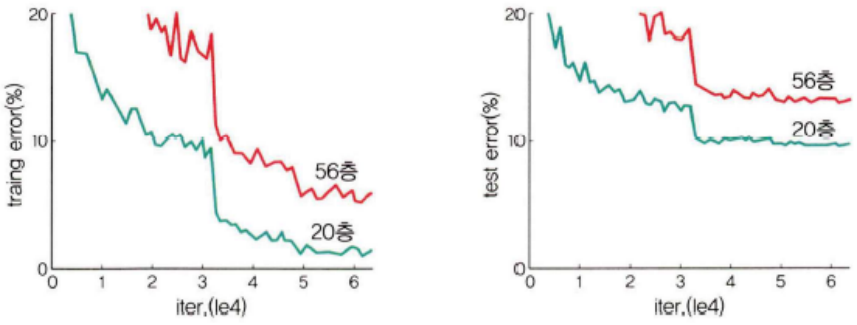
Week6_예습과제_진예원

6.1.5 ResNet

- 마이크로소프트에서 개발한 알고리즘. “Deep Residual Learning for Image Recognition” 논문에서 발표됨
- 핵심 개념: 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual) 개념을 고안
- 신경망 깊이와 성능 문제
  - 일반적으로 신경망 깊이가 깊어질수록 딥러닝 성능이 좋아질 것 같지만, 실상은 그렇지 x
  - 신경망은 깊이가 깊어질수록 성능이 좋아지다가, 일정한 단계에 다다르면 오히려 성능이 나빠짐(예: 56층이 20층보다 더 나쁜 성능을 보임)

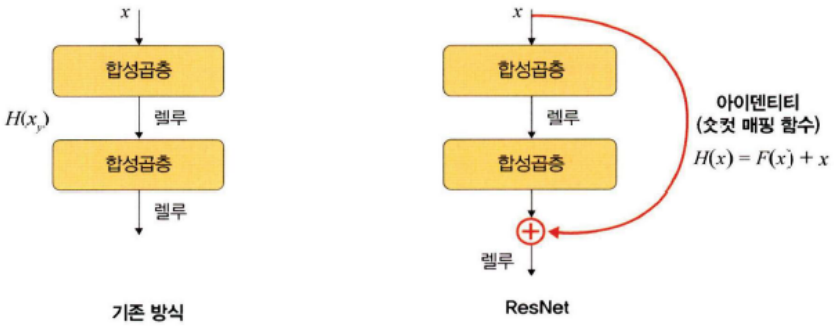
  ∴ 네트워크 깊이가 깊다고 해서 무조건 성능이 좋아지지는 않는다

그림 6-24 네트워크 56층이 20층보다 더 나쁜 성능을 보임



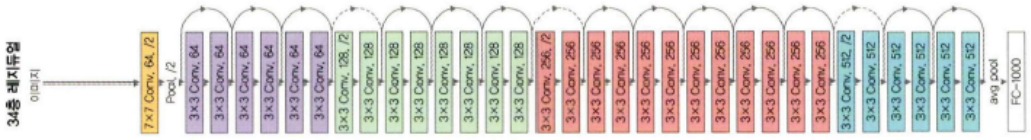
- 레지듀얼 블록(residual block)과 숏컷(shortcut)
  - ResNet은 이러한 성능 저하 문제 해결
  - 레지듀얼 블록 - 기울기가 잘 전파될 수 있도록 일종의 숏컷(shortcut, skip connection)을 만들어 줌
  - 2014년 공개된 GoogleNet은 층이 총 22개로 구성된 것에 비해, ResNet은 총 152개로 구성되어 기울기 소멸 문제가 발생할 수 있으므로, 숏컷을 두어 기울기 소멸 문제를 방지했음

❯ 그림 6-25 ResNet 구조



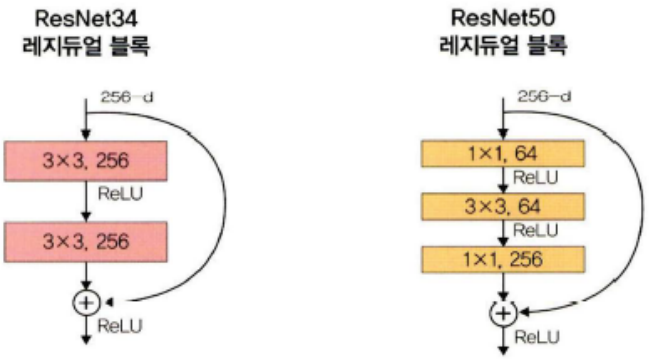
- 블록(block) 및 레지듀얼 블록 개념
  - 블록: 계층의 묶음, 합성곱층을 하나의 블록으로 묶은 것
  - 이렇게 묶인 계층들을 하나의 레지듀얼 블록이라고 함
  - 레지듀얼 블록을 여러 개 쌓은 것을 ResNet이라고 함
  - 합성곱층 > 레지듀얼 블록 > ResNet

❯ 그림 6-26 ResNet 모델 전체 네트워크



- 병목 블록(bottleneck block)
  - 계층의 깊이가 깊어질수록 파라미터는 무제한으로 커짐
  - 예: ResNet34 : 합성곱층이 34개와 16개의 블록으로 구성되어 있음. 첫 번째 블록의 파라미터가 1162k라면 전체 파라미터 수는 2만 1282k
  - 이러한 문제 해결을 위해 병목 블록을 사용
  - ResNet34 - 기본 블록(basic block) 사용 - 파라미터 수 39,3216M
  - ResNet50 - 병목 블록 사용 - 파라미터 수 6,9632M
  - 깊이가 깊어졌음에도 파라미터 수는 감소한 것!

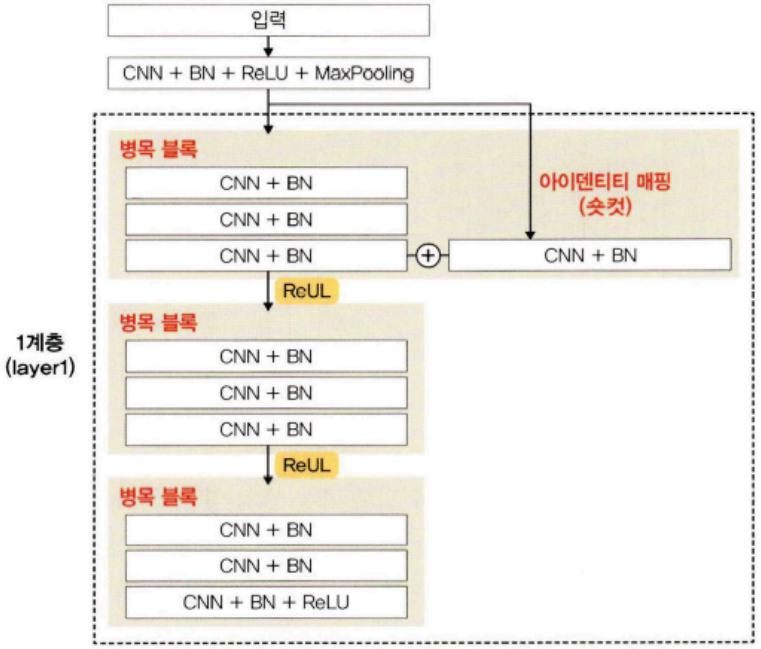
❯ 그림 6-27 기본 블록과 병목 블록



  - 병목 블록 원리: ResNet34와는 다르게 ResNet59에서는 3 x 3 합성곱층 앞뒤로 1 x 1 합성곱층이 붙어 있음. 1 x 1 합성곱층의 채널 수를 조절하면서 차원을 줄였다 늘리는 것이 가능하기 때문에 → 깊이가 깊어져도 파라미터 수를 줄일 수 있음

- 아이덴티티 매핑(identity mapping)(혹은 숏컷(shortcut), 스킵 연결(skip connection))
  - 위 그림 아래쪽에 + 기호 = 아이덴티티 매핑
  - 입력 x가 어던 함수를 통과하더라도 다시 x라는 형태로 출력되도록 함

❯ 그림 6-28 아이덴티티 매핑(숏컷)



- 구현 코드 중 forward() 맛보기

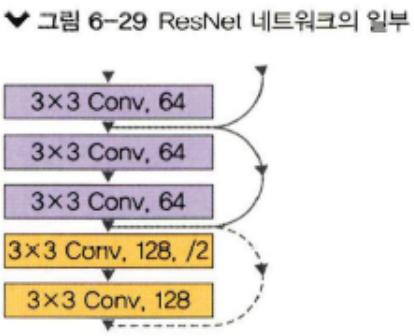
```
def forward(self, x):
    i = x
```

```
    x = self.conv(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2d(x)
    x = self.bn2(x)

    if self.downsample is not None:
        i = self.downsample(i) #다운샘플 적용

    x += I #아이덴티티 매핑 적용
    x = self.relu(x)
    return x
```

- 입력 x를 i라는 변수에 저장 / 입력 x는 합성곱층을 통과하다가 마지막 x에 I를 더해줌
  - 예: x가 (28, 28, 64) x를 i 변수에 저장. 합성곱층 통과하며 같은 형태를 더함 → 둘 다 (28, 28, 64) 그대로
- 다운샘플(downsample) - 특성 맵 크기를 줄이기 위한 것 = 풀링 역할



  - 보라색 영역
    - 첫 번째 블록에서 특성 맵 형상 (28, 28, 64)였다면 세 번째 블록의 마지막 합성곱층 통과하고 아이덴티티 매핑까지 완료된 특성 맵 형상도 (28, 28, 64)
  - 노란색 영역
    - 시작 시점에서는 채널 수가 128로 늘어남. /2 - 첫 번째 블록에서 합성곱층의 스트라이드가 2로 늘어나 (14, 14, 128)로 바뀜
  - 보라색과 노란색의 형태가 다른데 이들의 형태를 맞추지 않으면 아이덴티티 매핑 할 수 없게 됨 ⇒ 아이덴티티에 대해 다운 샘플 필요

- 입력 = 출력 형태 맞추기 위해선 스트라이드 2를 가진 1 x 1 합성곱 계층을 하나 연결해주면 됨
- 아이덴티티 블록: 입력과 출력의 차원이 같은 것
- 프로젝션 숏컷(projection-shortcut) 혹은 합성곱 블록: 입력 및 출력 차원이 동일하지 않고, 입력의 차원을 출력에 맞추어 변경해야 하는 것

그림 6-30 합성곱 블록

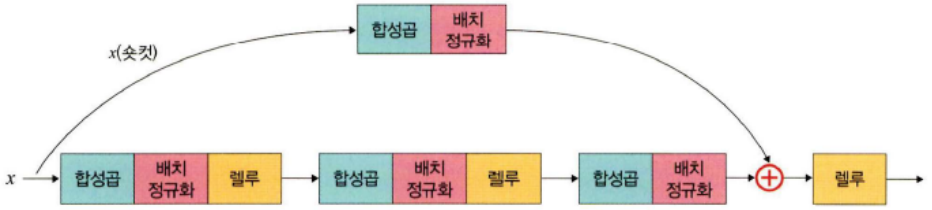
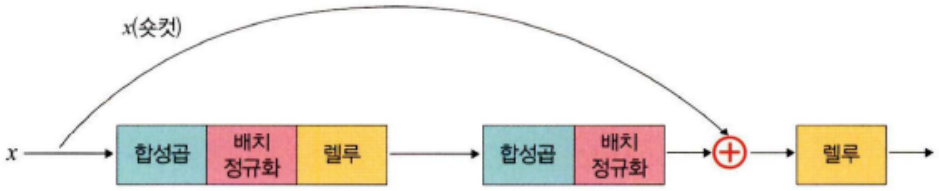
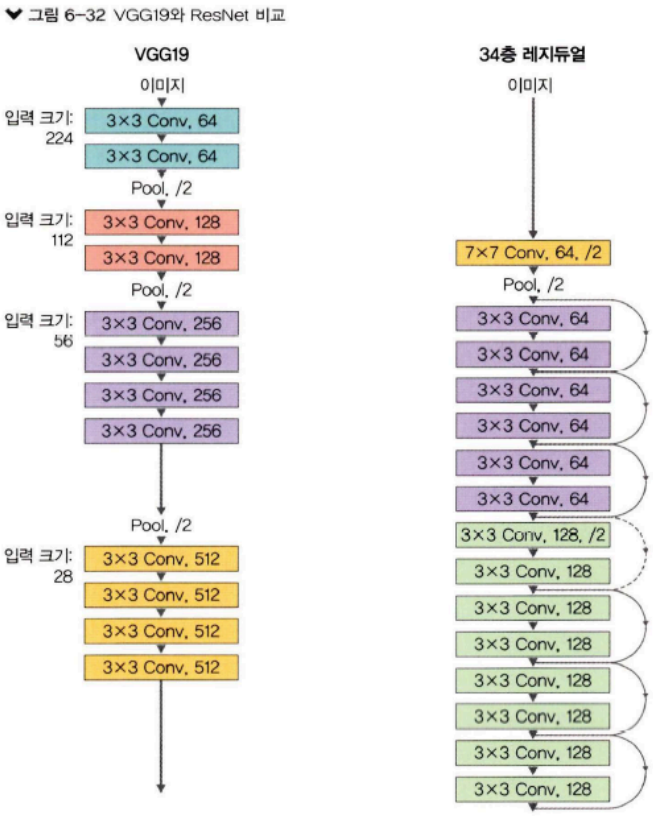


그림 6-31 아이덴티티 블록



- ResNet의 기본 뼈대 : VGG19 / 거기에 합성곱층들을 추가해서 깊게 만든 후 숏컷들을 추가하는 것



1. 필요한 라이브러리 호출

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as datasets
import torchvision.models as models

import matplotlib.pyplot as plt
import numpy as np

import copy
from collections import namedtuple #1
import os
import random
```

```
import time

import cv2
from torch.utils.data import DataLoader, Dataset
from PIL import Image

device = torch.device('cuda' if torch.cuda.is_available() e
lse 'cpu')
```

- 네임드튜플(namedtuple) : 파이썬의 자료형 중 하나. 튜플 성질 O + 인덱스뿐만 아니라 키 값으로 데이터 접근 가능

```
from collections import namedtuple
Student = namedtuple('Student', ['name', 'age', 'DOB'])
#네임드튜플 정의
S = Student('홍길동', '19', '187') #네임드튜플에 값 추가

print("The Student age using index is : ", end="")
print(S[1]) #인덱스 이용한 데이터 접근
print("The Student name using keyname is : ", end="")
print(S.name) #키 값 이용한 데이터 접근
```

  - 출력 결과

    The Student age using index is : 19
    The Student name using keyname is : 홍길동

2. 이미지 데이터 전처리

```
class ImageTransform():
  def __init__(self, resize, mean, std):
    self.data_transform = {
        'train': transforms.Compose([
            transforms.RandomResizedCrop(resize, scale=(0.
5, 1.0)),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
```

```
        ]), #훈련이미지 데이터에 대한 전처리
        'val': transforms.Compose([
            transforms.Resize(256),
            transforms.CenterCrop(resize),
            transforms.ToTensor(),
            transforms.Normalize(mean, std)
        ]) #검증과 테스트 이미지 데이터에 대한 전처리
    }

  def __call__(self, img, phase):
    return self.data_transform[phase](img)
```

3. 변수에 대한 값 정의

```
size = 224
mean = (0.485, 0.456, 0.406)
std = (0.229, 0.224, 0.225)
batch_size = 32
```

4. 훈련과 테스트 데이터셋 불러오기

- torchvision.datasets.ImageFolder 이용

```
cat_directory = r'../chap06/data/dogs-vs-cats/Cat/'
dog_directory = r'../chap06/data/dogs-vs-cats/Dog/'

cat_images_filepaths = sorted([os.path.join(cat_directory,
f) for f in os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory,
f) for f in os.listdir(dog_directory)])
images_filepaths = [*cat_images_filepaths, *dog_images_file
paths]
correct_images_filepaths = [i for i in images_filepaths if
cv2.imread(i) is not None]
```

- 출력 결과

400 92 10

- 훈련 데이터셋 400개, 검증 데이터셋 92개, 테스트 데이터셋 10개로 구성

5. 이미지에 대한 레이블 구분

- 가져온 데이터가 개 - 레이블 '1' 부여
- 가져온 데이터가 고양이 - 레이블 '0' 부여

```
class DogvsCatDataset(Dataset):
  def __init__(self, file_list, transform=None, phase='trai
n'):
    self.file_list = file_list
    self.transform = transform
    self.phase = phase

  def __len__(self):
    return len(self.file_list)

  def __getitem__(self, idx):
    img_path = self.file_list[idx]
    img = Image.open(img_path)
    img_transformed = self.transform(img, self.phase)

    label = img_path.split('/')[-1].split('.')[0]
    if label == 'dog':
      label = 1
    elif label == 'cat':
      label = 0
    return img_transformed, label
```

6. 이미지 데이터셋 정의

```
train_dataset = DogvsCatDataset(train_images_filepaths,
                                transform=ImageTransform(si
ze, mean, std), phase='train')
val_dataset = DogvsCatDataset(val_images_filepaths,
                              transform=ImageTransform(siz
```

```
e, mean, std), phase='val')
index = 0
print(train_dataset.__getitem__(index)[0].size())
print(train_dataset.__getitem__(index)[1])
```

- 출력 결과

  torch.Size([3, 224, 224])
  0

  - 이미지는 컬러(채널3) 상태에서 224 x 224 크기를 갖고 있음
  - 레이블이 0 → 고양이 의미

7. 데이터셋의 데이터를 메모리로 불러오기

```
train_iterator = DataLoader(train_dataset, batch_size=batch
_size, shuffle=True)
valid_iterator = DataLoader(val_dataset, batch_size=batch_s
ize, shuffle=False)
dataloader_dic = {'train': train_iterator, 'val': valid_ite
rator}
batch_iterator = iter(train_iterator)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

- 출력 결과

  torch.Size([32, 3, 224, 224])
  tensor([1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 1, 1,
  1, 0, 1, 0, 1, 0, 0, 0])

  - 데이터로더를 이용하여 메모리로 불러온 훈련 데이터셋의 이미지 크기 & 레이블

8. 기본 블록 정의

- ResNet 전체 네트워크 구성 위해 그것을 구성하는 기본 블록과 병목 블록에 대한 코드 작성
- 기본 블록

- ResNet18, ResNet34에서 사용
- 합성곱(3x3) 두 개로 구성

```
class BasicBlock(nn.Module):
  expansion = 1

  def __init__(self, in_channels, out_channels, stride=1, d
ownsample=False):
    super().__init__()
    self.cov1 = nn.Conv2d(in_channels, out_channels, kernel
_size=3,
                          stride=stride, padding=1, bias=Fa
lse) # 3x3 합성곱층
    self.bn1 = nn.BatchNorm2d(out_channels)
    self.conv2 = nn.Conv2d(out_channels, out_channels, kern
el_size=3,
                           stride=1, padding=1, bias=False)
#3x3 합성곱층
    self.bn2 = nn.BatchNorm2d(out_channels)
    self.relu = nn.ReLU(inplace=True)

    if downsample: #1
      conv = nn.Conv2d(in_channels, out_channels, kernel_si
ze=1,
                       stride=stride, bias=False)
      bn = nn.BatchNorm2d(out_channels)
      downsample = nn.Sequential(conv, bn)
    else:
      downsample = None
    self.downsample = downsample

  def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)
```

```
    if self.downsample is not None:
      i = self.downsample(i)

    x += I  #2
    x = self.relu(x)

    return x
```

- 다운 샘플
  - 입력 데이터의 크기와 네트워크를 통과한 후 출력 데이터의 크기가 다를 경우 사용
  - 다운샘플 위해서 합성곱층에 스트라이드 적용함
    (conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=False)
- 아이덴티티 매핑
  - 특정 층에 존재하는 출력 결과를 다음 합성곱층을 통과한 출력 결과에 더해줌 = 스킵 연결(skip connection)이라고도 함
  - 예: x에 conv1, bn1, relu, conv2, bn2, relu, conv3, bn3 값이 더해지다가, 초기의 x가 다시 더해지는 것을 아이덴티티 매핑 = 숏컷 = 스킵 연결이라고 함

```
def forward(self, x):
  i = x
  x = self.conv1(x)
  x = self.bn1(x)
  x = self.relu(x)
  x = self.conv2(x)
  x = self.bn2(x)
  x = self.relu(x)
  x = self.conv3(x)
  x = self.bn3(x)

  if self.downsample is not None:
    i = self.downsample(i)

  x += i
  x = self.relu(x)
```

```
    return x
```

9. 병목 블록 정의

- 병목 블록
  - ResNet50, ResNet101, ResNet152에서 사용
  - 1 × 1 합성곱층, 3 x 3 합성곱층, 1 x 1 합성곱층으로 구성

```
class Bottleneck(nn.Module):
  expansion = 4 #병목블록 정의 위한 하이퍼파라미터

  def __init__(self, in_channels, out_channels, stride=1, d
ownsample=False):
    super().__init__()
    self.conv1 = nn.Conv2d(in_channels, out_channels, kerne
l_size=1,
                           stride=1, bias=Fasle) #1x1 합성곱
층
    self.bn1 = nn.BatchNorm2d(out_channels)
    self.conv2 = nn.Conv2d(out_channels, out_channels, kern
el_size=3,
                           stride=stride, padding=1, bias=F
asle) #3x3 합성곱층
    self.bn2 = nn.BatchNorm2d(out_channels)
    self.conv3 = nn.Conv2d(out_channels, self.expansion*out
_channels,
                           kernel_size=1, stride=1, bias=Fa
sle) #1x1 합성곱층
    self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
    self.relu = nn.ReLU(inplace=True)

    if downsample:
      conv = nn.Conv2d(in_channels, self.expansion*out_chan
nels, kernel_size=1,
                       stride=stride, bias=False)
      bn = nn.BatchNorm2d(self.expansion*out_channels)
```

```
      downsample = nn.Sequential(conv, bn)
    else:
      downsample = None
    self.downsample = downsample

  def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)
    x = self.relu(x)
    x = self.conv3(x)
    x = self.bn3(x)

    if self.downsample is not None:
      i = self.downsample(i)

    x += i
    x = self.relu(x)

    return x
```

- 기본 블록과 병목 블록 차이
  - 기본 블록 - 3x3 합성곱층 두 개 가짐
  - 병목 블록 - 1x1 합성곱층, 3x3 합성곱층, 1x1 합성곱층 구조 가짐
- 기본 블록 → 병목 블록 변경 이유: 계층을 더 깊게 쌓으면서 계산에 대한 비용 줄일 수 있음
  - 계층이 많아진다는 뜻 = 곧 활성화 함수가 기존보다 더 많이 포함된다 = 더 많은 비선형성 처리할 수 있음 의미

  → 다양한 입력 데이터에 대한 처리 가능
- ∴ 아이덴티티 매핑 + 병목 블록 → ResNet 네트워크에 더 깊은 계층 쌓을 수 있게 됨

10. ResNet 모델 네트워크

```
class ResNet(nn.Module):
  def __init__(self, config, output_dim, zero_init_residual
=False):
    super().__init__()
    block, n_blocks, channels = config
    self.in_channels = channels[0]
    assert len(n_blocks) == len(channels) == 4
    self.conv1 = nn.Conv2d(3, self.in_channels, kernel_size
=7, stride=2,
                           padding=3, bias=False)
    self.bn1 = nn.BatchNorm2d(self.in_channels)
    self.relu = nn.ReLU(inplace=True)
    self.maxpool = nn.MaxPool2d(kernel_size=3, stride=2, pa
dding=1)

    self.layer1 = self.get_resnet_layer(block, n_blocks[0],
channels[0])
    self.layer2 = self.get_resnet_layer(block, n_blocks[1],
channels[1], stride=2)
    self.layer3 = self.get_resnet_layer(block, n_blocks[2],
channels[2], stride=2)
    self.layer4 = self.get_resnet_layer(block, n_blocks[3],
channels[3], stride=2)

    self.avgpool = nn.AdaptiveAvgPool2d((1,1))
    self.fc = nn.Linear(self.in_channels, output_dim)

    if zero_init_residual: #1
      for m in self.modules():
        if isinstance(m, Bottelneck):
          nn.init.constant_(m.bn3.weight, 0)
        elif isinstance(m, BasicBlock):
          nn.init.constant_(m.bn2.weight, 0)

  def get_resnet_layer(self, block, n_blocks, channels, str
ide=1):
    layers = []
    if self.in_channels != block.expansion * channels:
```

```
      downsample = True
    else:
      downsample = False

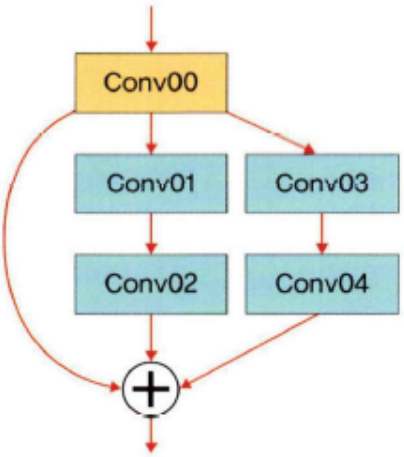
    layer.append(block(self.in_channels, channels, stride,
downsample))
    for i in range(1, n_blocks):
      layers.append(block(block.expansion*channels, channel
s))

    self.in_channels = block.expansion * channels
    return nn.Sequential(*layers)

  def forward(self, x):
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.maxpool(x)
    x = self.layer1(x)
    x = self.layer2(x)
    x = self.layer3(x)
    x = self.layer4(x)
    x = self.avgpool(x)
    h = x.view(x.shape[0], -1)
    x = self.fc(h)
    return x, h
```

- 각 레지듀얼 분기(residual branch)에 있는 마지막 BN(Batch Normalization)을 0으로 초기화해서 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함
  - 이 부분은 모델 생성, 학습시키는 것과는 상관 X
  - But BN 0으로 초기화할 경우 모델 성능 0.2 ~ 0.3% 정도 향상됨
- 레지듀얼 분기: 프로그램에서의 조건에 따라 A, B, C 등으로 분기하는 것과 같음

❯ 그림 6-33 레지듀얼 분기



11. ResNetConfig 정의

- ResNetConfig 변수에 네임드 튜플 데이터 형식으로 ['block', 'n_blocks', 'channels'] 를 저장

```
ResNetConfig = namedtuple('ResNetConfig', ['block', 'n_blocks', 'channels'])
```

12. 기본 블록을 사용하여 ResNetConfig 정의

- 먼저 기본 블록 사용하는 ResNet18과 ResNet34의 Config를 정의
- 블록은 기본 블록(BasicBlock) 사용 / 블록(n_blocks)과 채널(channels) 크기 각각 지정

```
resnet18_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[2,2,2,2],
                               channels=[64,128,256,512])

resnet34_config = ResNetConfig(block=BasicBlock,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])
```

13. 병목 블록 사용하여 ResNetConfig 정의

- 다음으로 병목 블록 사용하는 ResNet50, ResNet101, ResNet152의 Config 정의

- 블록은 병목 블록(Bottleneck) 사용 / 블록(n_blocks)과 채널(channels) 크기 각각 지정

```
resnet50_config = ResNetConfig(block=Bottleneck,
                               n_blocks=[3,4,6,3],
                               channels=[64,128,256,512])

resnet101_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,4,23,3],
                                channels=[64,128,256,512])

resnet152_config = ResNetConfig(block=Bottleneck,
                                n_blocks=[3,8,36,3],
                                channels=[64,128,256,512])
```

14. 사전 훈련된 ResNet 모델 사용

- 아이덴티티 매핑, 다운샘플 및 명복 블록까지 정의하려니 번거로움

→ 사전 훈련된 모델이므로 코드 한 줄로 간단하게 사용 가능

```
pretrained_model = models.resnet50(pretrained=True)
```

15. 사전 훈련된 ResNet 네트워크 확인

- 우리가 만든 ResNet 네트워크 = 사전 훈련된 ResNet 네트워크인지 확인

```
print(pretrained_model)
```

16. ResNet50 Config를 사용한 ResNet 모델 사용

- ResNet50 모델 사용 위해 Config 부분에 resnet50_config를 지정하고 OUTPUT_DIM에는 테스트 데이터셋의 클래스(개와 고양이)를 입력함

```
OUTPUT_DIM = 2
model = ResNet(resnet50_config, OUTPUT_DIM)
print(model)
```

- 15와 16 출력 결과와 비교해보자 → 동일함

17. 옵티마이저와 손실함수 정의

```
optimizer = optim.Adam(model.parameters(), lr=1e-7)
criterion = nn.CrossEntropyLoss()
model = model.to(device)
criterion = criterion.to(device)
```

- lr = 1e-7 : 1*10의 -7승

18. 모델 학습 정확도 측정 함수 정의

```
def calculate_topk_accuracy(y_pred, y, k=2):
    with torch.no_grad():
        batch_size = y.shape[0]
        _, top_pred = y_pred.topk(k, 1) #1
        top_pred = top_pred.t() #2
        correct = top_pred.eq(y.view(1, -1).expand_as(top_p
red)) #3
        correct_1 = correct[:1].reshape(-1).float().sum(0,
keepdim=True)
        correct_k = correct[:k].reshape(-1).float().sum(0,
keepdim=True)
        acc_1 = correct_1 / batch_size
        acc_k = correct_k / batch_size
    return acc_1, acc_k
```

- tensor.topk = torch.argmax : 주어진 텐서에서 가장 큰 값의 인덱스 얻음

  → 네트워크 출력에서 가장 확률이 높은 값의 인덱스 반환

  - 예시

```
import torch
x = torch.arange(1., 6.)
print(x)
print('----------')
```

```
print(torch.topk(x, 3)) #x 입력에서 가장 큰 값 세게 선택하여 그 값과 인덱스 출력
```

- 출력 결과

  tensor([1., 2., 3., 4., 5.]
  torch.return_types.topk(
  values=tensor([5., 4., 3.]),
  indices=tensor([4, 3, 2]))

- t(): 차원 0과 1을 전치(transpose)
  - 예시

```
x = torch.rand(3)
print(x)
print(torch.t(x))
print('---------')
x = torch.randn(3, 2) #(2,3) 차원 텐서 생성
print(x)
print(torch.t(x)) #입력이 전치되어 (3,2) 차원 반환
```

  - 출력 결과

    tensor([0.6969, 0.8874, 0.4615])
    tensor([0.6969, 0.8874, 0.4615])
    tensor([[0.5666, 0.2586],
    [0.6017, 0.2483],
    [-2.0452, -0.8806]])
    tensor([[0.5666, 0.6017, -2.0452],
    [0.2586, 0.2483, -0.8806]])

- torch.eq: 텐서를 비교하는 함수
  - 텐서가 서로 같은지를 비교 → torch.eq
  - 텐서가 서로 다른지를 비교 → torch.ne
  - 텐서가 서로 크거나 같은지를 비교 → torch.ge
  - 형식: torch.eq(비교 대상 텐서, 비교할 텐서)
  - 결과: 각 텐서의 요소들 비교해서 같으면 True, 다르면 False 반환
  - 예시

```
torch.eq(torch.tensor([[1, 2], [3, 4]]), torch.tensor
([[1, 1], [4, 4]]))
```

- 출력 결과

  tensor([[True, False],
  [False, True]])

- 1과 1을 비교했을 때 서로 같으므로 True, .. 이렇게 각 텐서의 요소들 각각 비교

19. 모델 학습 함수 정의

```
def train(model, iterator, optimizer, criterion, scheduler,
device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.train()
    for (x, y) in iterator:
        x = x.to(device)
        y = y.to(device)

        optimizer.zero_grad()
        y_pred = model(x)
        loss = criterion(y_pred[0], y)

        acc_1, acc_5 = calculate_topk_accuracy(y_pred[0],
y)
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        epoch_acc_1 += acc_1.item()
        epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
```

```
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

20. 모델 평가 함수 정의

```
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc_1 = 0
    epoch_acc_5 = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred = model(x)
            loss = criterion(y_pred[0], y)

            acc_1, acc_5 = calculate_topk_accuracy(y_pred
[0], y)
            epoch_loss += loss.item()
            epoch_acc_1 += acc_1.item()
            epoch_acc_5 += acc_5.item()

    epoch_loss /= len(iterator)
    epoch_acc_1 /= len(iterator)
    epoch_acc_5 /= len(iterator)
    return epoch_loss, epoch_acc_1, epoch_acc_5
```

21. 모델 학습 시간 측정 함수 정의

```
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)
```

```
    elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
    return elapsed_mins, elapsed_secs
```

22. 모델 학습

```
best_valid_loss = float('inf')
EPOCHS = 10

scheduler = None

for epoch in range(EPOCHS):
    start_time = time.monotonic()

    # criterion과 scheduler의 순서 맞춤 (optimizer -> criterion -> scheduler -> device)
    train_loss, train_acc_1, train_acc_5 = train(model, train_iterator, optimizer, criterion, scheduler, device)
    valid_loss, valid_acc_1, valid_acc_5 = evaluate(model, valid_iterator, criterion, device)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), './chap06/data/ResNet-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time)

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc @1: {train_acc_1*100:.2f}% | Train Acc @5: {train_acc_5*100:.2f}%')
    print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc @1: {valid_acc_1*100:.2f}% | Valid Acc @5: {valid_acc_5*100:.2f}%')
```

- 모델 학습 결과에 대한 오차와 정확도 측면에서 성능이 좋진 않음
- 이미지 데이터 늘리면 성능 좋아질 수 있음

23. 테스트 데이터셋을 이용한 모델 예측

- 모델이 얼마나 잘 학습되었는지 알아보기 위해 테스트 데이터셋을 이용하여 예측 결과를 ResNet, csv로 저장

```
import pandas as pd
id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    for test_path in test_images_filepaths:
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
        transform = ImageTransform(size, mean, std)
        img = transform(img, phase='val')
        img = img.unsqueeze(0)
        img = img.to(device)

        model.eval()
        outputs = model(img)
        preds = F.softmax(outputs[0], dim=1)[:, 1].tolist()
        id_list.append(_id)
        pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.sort_values(by='id', inplace=True)
res.reset_index(drop=True, inplace=True)

res.to_csv('./chap06/data/ResNet.csv', index=False)
res.head(10)
```

	id	label
0	109	0.631571
1	145	0.620124
2	15	0.599507
3	162	0.551198
4	167	0.510797
5	200	0.624055
6	210	0.591528
7	211	0.578030
8	213	0.496708
9	224	0.587175

- 레이블 > 0.5 → 개 / < 0.5 → 고양이

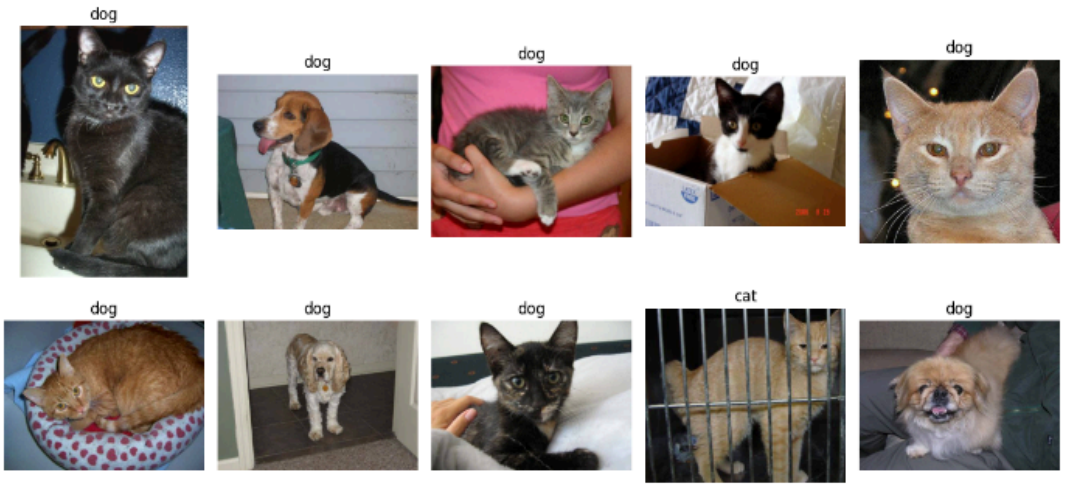
24. 모델 예측에 대한 결과 출력

```
class_ = classes = {0:'cat', 1:'dog'}
def display_image_grid(images_filepaths, predicted_labels=
(), cols=5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows=rows, ncols=cols, figsi
ze=(12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

        a = random.choice(res['id'].values)
        label = res.loc[res['id'] == a, 'label'].values[0]

        if label > 0.5:
            label = 1
        else:
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
```

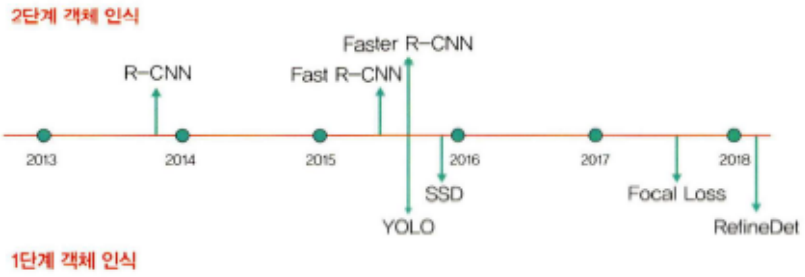
```
            ax.ravel()[i].set_axis_off()
        plt.tight_layout()
        plt.show()
display_image_grid(test_images_filepaths)
```



- 모델 학습 결과에 대한 오차와 정확도 측면에서 성능이 좋진 않음
- 이미지 데이터 늘리면 성능 좋아질 수 있음

6.2 객체 인식을 위한 신경망

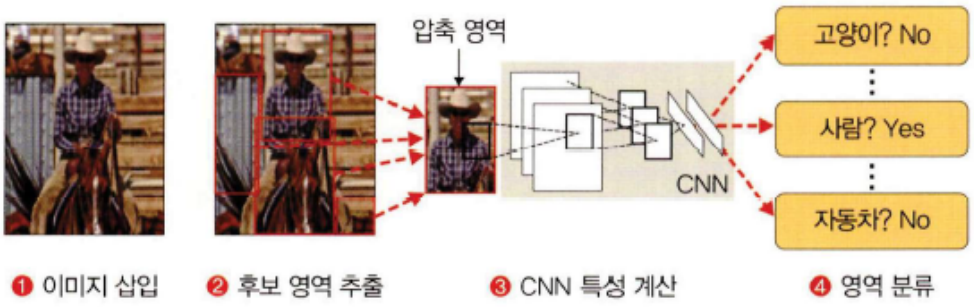
- 객체 인식(object detection) : 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
    - 이미지나 영상 내에 있는 여러 객체에 대해 각 객체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스(bounding box)로 나타내는 위치 검출(localization) 문제를 다루는 분야
    - 객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출로 표현 가능
    - 자율 주행 자동차, CCTV, 무인 점포 등 많은 곳에서 활용됨
- 딥러닝을 이용한 객체 인식 알고리즘



    - 1단계 객체 인식
        - 분류와 위치 검출을 동시에 행하는 방법
        - 비교적 빠르지만, 정확도가 낮음
        - YOLO(You Only Look Once) 계열과 SSD 계열 등이 포함됨
    - 2단계 객체 인식
        - 분류와 위치 검출을 순차적으로 행하는 방법
        - 비교적 느리지만, 정확도가 높음
        - CNN을 처음으로 적용시킨 R-CNN 계열이 대표적
- 여기서는 2단계 객체 인식 알고리즘을 알아볼 예정

6.2.1 R-CNN

- 예전 객체 인식 알고리즘 - 슬라이딩 윈도우(sliding window) 방식
    - 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해 내는 방식
    - 알고리즘의 비효율성으로 많이 사용 X
- 현재 객체 인식 알고리즘 - 선택적 탐색(selective search) 알고리즘을 적용한 후보 영역(region proposal) 사용
- R-CNN(Region-based CNN) : 이미지 분류를 수행하는 CNN과 이미지에서 객체가 있을 만한 영역을 제안해 주는 후보 영역 알고리즘을 결합한 알고리즘
- R-CNN 수행 과정



1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 선택적 알고리즘으로 추출한 후 잘라내고(copying), CNN 모델에 넣기 위해 같은 크기(227 x 227 픽셀)로 통일함(warping)
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용함
4. 각각 분류를 진행하여 결과를 도출함

💡 선택적 탐색
: 객체 인식이나 검출을 위한 가능한 후보 영역(객체가 있을 만한 위치, 영역)을 알아내는 방법. 분할 방식을 이용하여 시드(seed)를 선정하고, 그 시드에 대한 완전 탐색을 적용함

1. 1단계 초기 영역 생성(sub-segmentation)
   - 각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기 영역을 생성함 → 입력된 이미지를 영역 다수 개로 분할하는 과정



2. 2단계 작은 영역의 통합
   - 1단계에서 영역 여러 개로 나눈 것들을 비슷한 영역으로 통합함
   - 탐욕(greedy) 알고리즘을 사용하여 비슷한 영역이 하나로 통합될 때까지 반복함



3. 3단계 후보 영역 생성
    - 2단계에서 통합된 이미지들을 기반으로 그림과 같이 후보 영역(바운딩 박스)을 추출



- 용어 의미
    - 완전 탐색(exhaustive search): 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황 고려하여 후보 영역 찾는 기법
    - 분할(segmentation): 영상 데이터의 특성(색상, 모양, 무늬 등)에 다라 분할하여 후보 영역을 선정하는 기법
    - 후보 영역(바운딩 박스): 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
    - 시드(seed): 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀

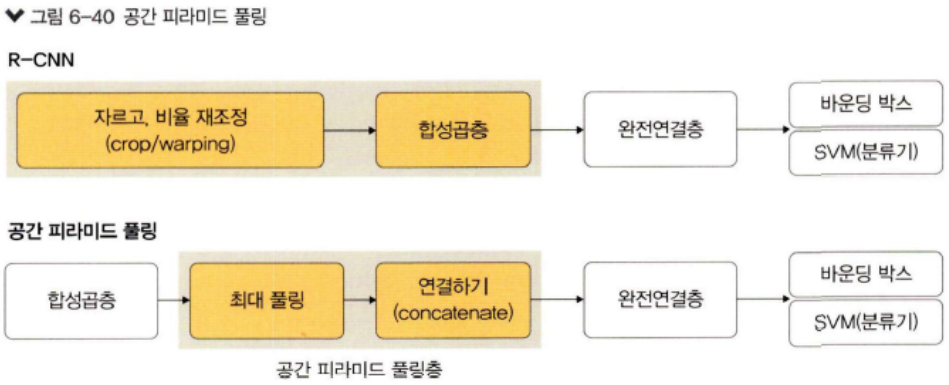
- R-CNN 단점
    1. 앞서 언급한 세 단계의 복잡한 학습 과정
    2. 긴 학습 시간과 대용량 저장 공간
    3. 객체 검층(object detection) 속도 문제

→ 문제 해결: Fast R-CNN

6.2.2 공간 피라미드 풀링

- 공간 피라미드 풀링 도입 배경
  - 기존 CNN 구조들은 모두 완전연결층을 위해 입력 이미지를 고정해야 했음
  - 그렇기 때문에 신경망 통과시키려면 이미지를 고정된 크기로 자르거나 비율을 조정해야했음
  - but 이렇게 하면 일부분이 잘리거나 본래의 생김새와 달라지는 문제 발생

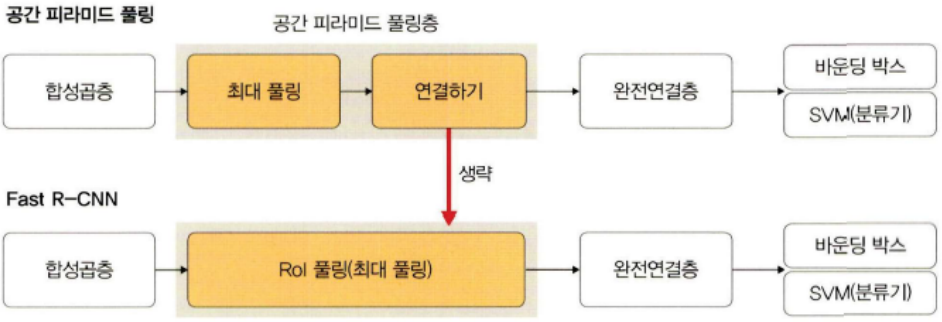
  → 이러한 문제 해결하고자 공간 피라미드 풀링(spatial pyramid pooling) 도입



- 공간 피라미드 풀링
  - 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, 완전연결층에 전달되기 전에 특성 맵들을 동일한 크기로 조절해 주는 풀링층을 적용하는 기법
  - 입력 이미지의 크기를 조절하지 않고 합성곱층 통과시키므로 원본 이미지의 특징 훼손되지 않는 특성 맵 얻을 수 있음
  - 이미지 분류나 객체 인식 같은 여러 작업에 적용할 수 있음

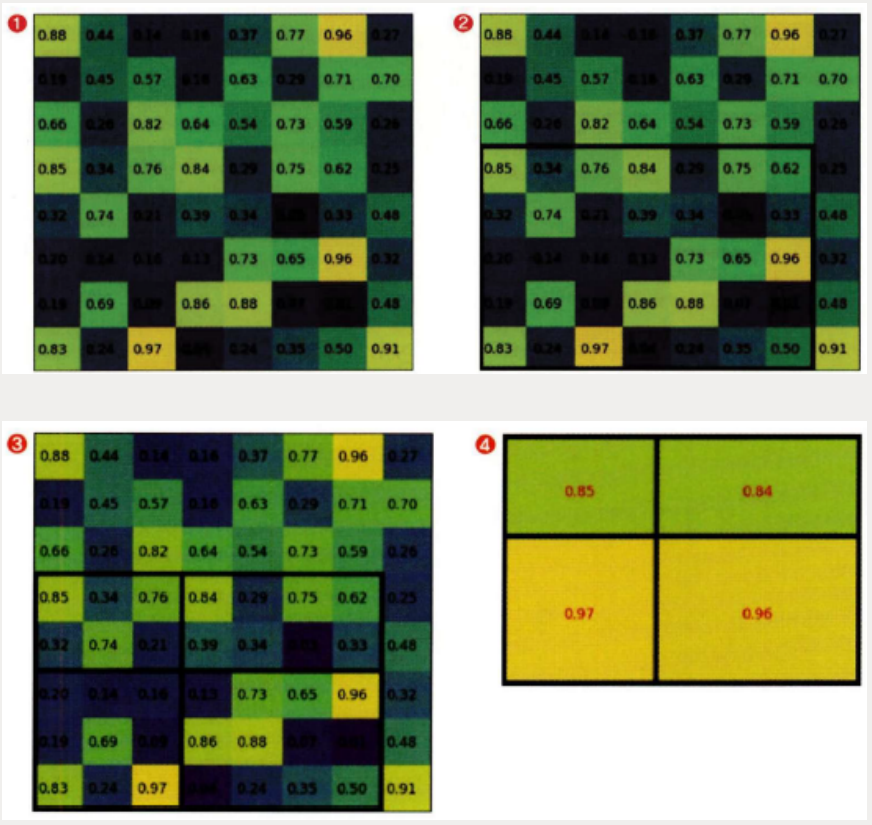
6.2.3 Fast R-CNN

- R-CNN - 바운딩 박스마다 CNN을 돌리고, 분류를 위한 긴 학습 시간이 문제
- Fast R-CNN - R-CNN의 속도 문제 개선하려고 RoI 풀링 도입
  - 선택적 탐색에서 찾은 바운딩 박스 정보가 CNN 통과하면서 유지되도록하고 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정함
    → 바운딩 박스마다 CNN을 돌리는 시간 단축할 수 있음



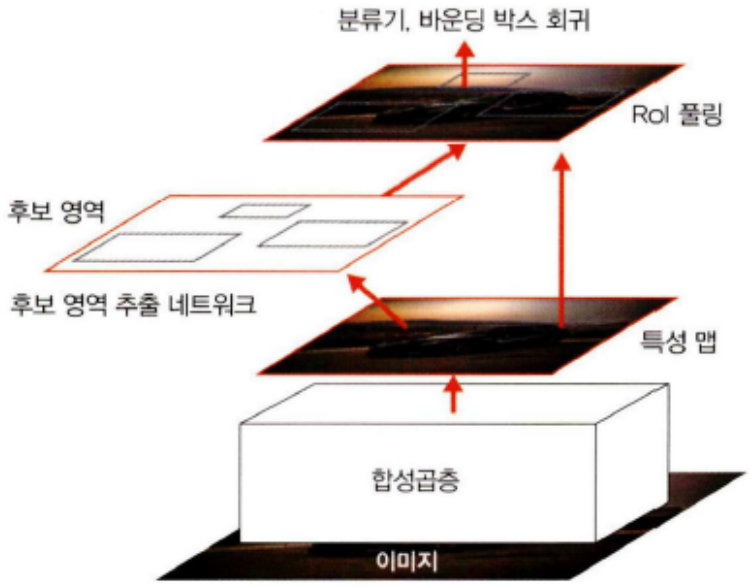
RoI 풀링
: 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결괏값 크기를 동일하게 맞추는 방법

- 예시: 박스 한 개가 픽셀 한 개를 뜻하는 특성 맵이 있다고 하자. 8x8 특성 맵(1)에서 선택적 탐색으로 뽑아냈던 7x5 후보 영역(2)이 있으며, 이것을 2x2로 만들기 위해 스트라이드(7/2=3, 5/2=2)로 풀링 영역(3)을 정하고 최대 풀링을 적용하면 2x2 결과(4)를 얻을 수 있다

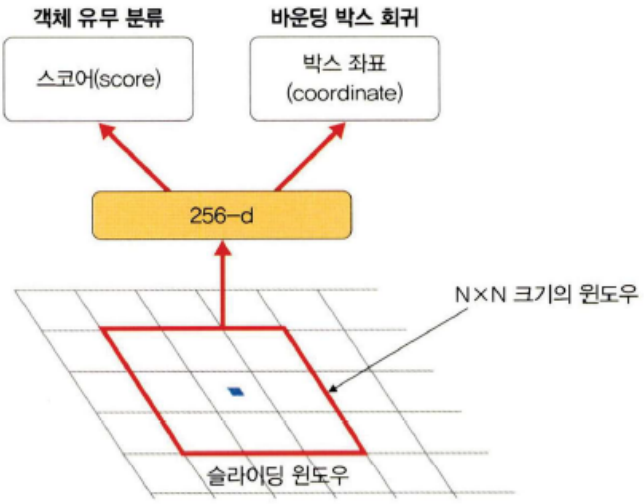


6.2.4 Faster R-CNN

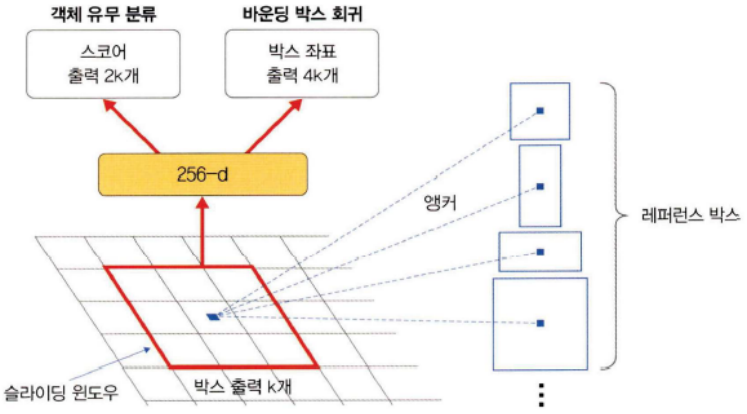
- 더욱 빠른 객체 인식을 수행하기 위한 네트워크
- Fast R-CNN과의 차이
  - 기존 Fast R-CNN 속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계
  - 기존 Fast R-CNN에 후보 영역 추출 네트워크를 추가
  - 외부이 느린 선택적 탐색(CPU로 계산) 대신 내부의 빠른 RPN(GPU로 계산)을 사용
- 구조
  - RPN - 마지막 합성곱층 다음에 위치
  - 그 뒤에 (Fast R-CNN과 일치) RoI 풀링과 분류기, 바운딩 박스 회귀가 위치



- 후보 영역 추출 네트워크 - 특성 맵 N x N 크기의 작은 윈도우 영역을 입력으로 받고, 해당 영역에 객체의 존재 유무 판단 위해 이진 분류를 수행하는 작은 네트워크 생성
  - 바운딩 박스 회귀 - 위치 보정(좌표점 추론)을 위해 추가
  - 슬라이딩 윈도우 방식 - 하나의 특성 맵에서 모든 영역에 대한 객체의 존재 유무 확인 위함. 앞서 설계한 작은 윈도우 영역(N x N 크기)을 이용하여 객체를 탐색함
  - 단점: 이미지에 존재하는 객체들의 크기와 비율 다양 → 고정된 N x N 크기의 입력만으로 다양한 크기와 비율의 이미지 수용 어려움



- 앵커(anchor) - 후보 영역 추출 네트워크의 고정된 입력 수용의 문제점을 보완하기 위해, 여러 크기와 비율의 레퍼런스 박스(reference box) k개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계
  - 후보 영역 추출 네트워크의 출력 값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 분류에 대한 출력 + x,y,w,h 위치 보정 값을 위한 4k개의 회귀 출력을 가짐
  - 예시: 특성 캡 크기 w x h → 하나의 특성 맵에 앵커가 총 w x h x k개 존재



6.3 이미지 분할을 위한 신경망

- 이미지 분할(image segmentation): 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것

  → 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출함

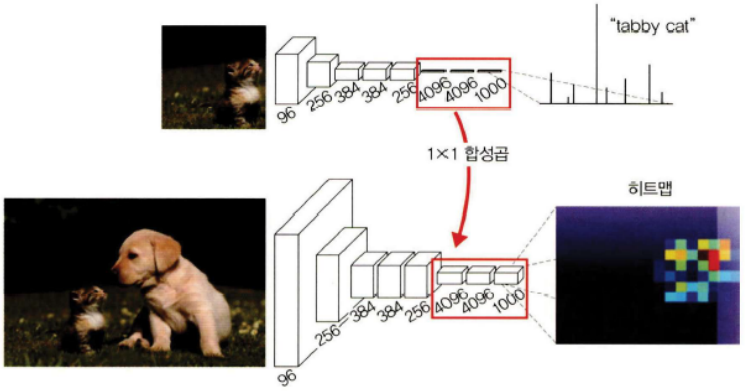
- 이미지 분할의 대표적 네트워크: 완전 합성곱 네트워크, 합성곱 & 역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/DeepLabv3+

6.3.1 완전 합성곱 네트워크

- 완전연결층의 한계: 고정된 크기의 입력만 받아들이며, 완전연결층을 거친 후에는 위치 정보가 사라짐

  → 해결책: 완전연결층을 1 x 1 합성곱으로 대체 = 완전 합성곱 네트워크

- 완전 합성곱 네트워크(Fully Convolutional Network, FCN): 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크
- 다음 그림과 같이 AlexNet 아래쪽에서 사용되었던 완전연결층 세 개를 1 x 1 합성곱으로 변환하면 → 위치 정보가 남아 있기 때문에 히트맵(heatmap) 그림과 같이 고양이의 위치를 확인할 수 있음
- 또, 합성곱층으로 사용되므로 입력 이미지에 대한 크기 제약이 사라지는 장점 가짐



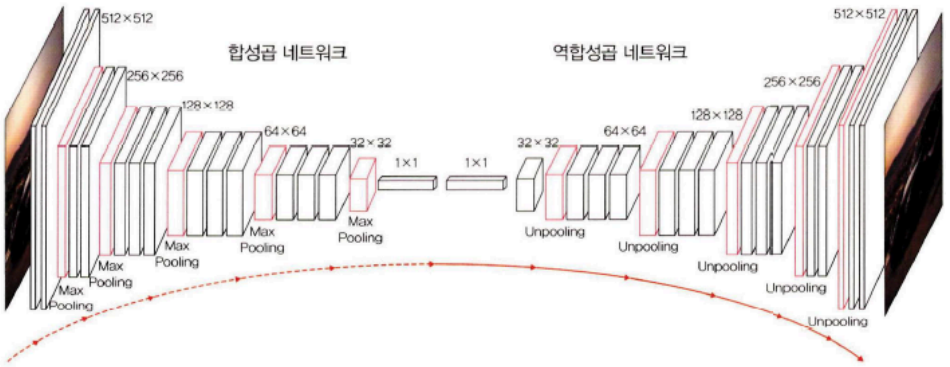
6.3.2 합성곱 & 역합성곱 네트워크

- 완전 합성곱 네트워크
  - 장점: 위치 정보 보존됨
  - 단점
    - 여러 단계의 합성곱층과 풀링층 거치면서 해상도가 낮아짐
    - 낮아진 해상도를 복원하기 위해 업 샘플링 방식을 사용하기 때문에 이미지의 세부 정보들을 잃어버리는 문제 발생

  → 해결책: 합성곱 & 역합성곱 네트워크(convolutional & deconvolutional network)
- 역합성곱 = 업 샘플링
  - CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을 때 사용

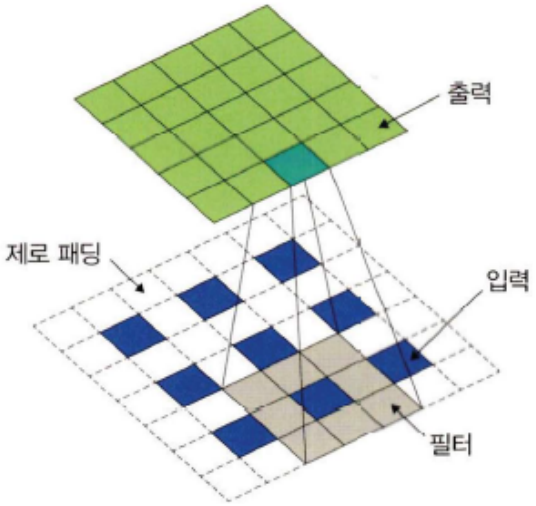
- 시멘틱 분할(semantic segmentation) 등에 활용

그림 6-47 합성곱 & 역합성곱 네트워크



- 역합성곱 동작 방식 (역합성곱은 합성곱층이 합성곱을 사용하여 특성 맵 크기를 줄이는 것과 반대로, 특성 맵 크기를 증가시키는 방식으로 동작함)
  1. 각각의 픽셀 주위에 제로 패딩(zero-padding)을 추가
  2. 이렇게 패딩된 것이 합성곱 연산 수행

그림 6-48 역합성곱 진행 방식

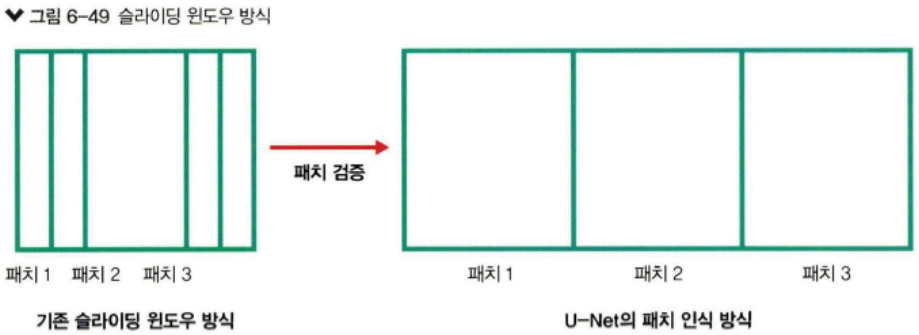


- 입력: 아래쪽의 파란색 픽셀
- 출력: 초록색 픽셀
- 파란색 픽셀 주위로 흰색 제로 패딩 수행 → 회색 필터로 합성곱 연산 수행 → 초록색 출력

6.3.3 U-Net

- 바이오 메디컬 이미지 분할을 위한 합성곱 신경망
- 특징
  - 속도가 빠름
    - 기존 슬라이딩 윈도우 방식 - 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하기 때문에 속도가 느렸음

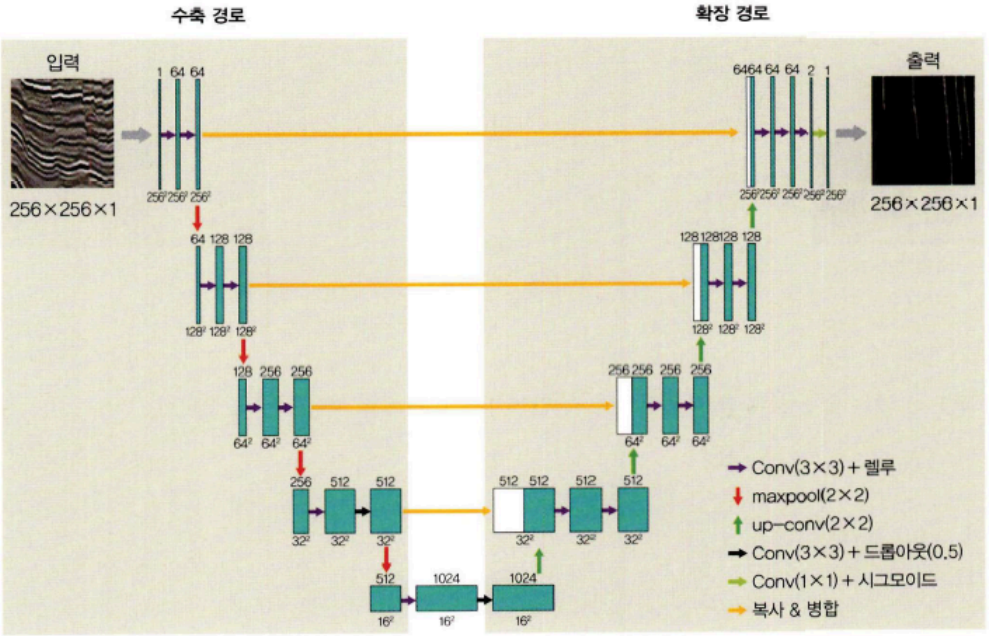
        - U-Net은 이미 검증 끝난 패치는 건너뛰기 때문에 속도가 빠름



    - 트레이드오프(trade-off)에 빠지지 않음
        - 일반적으로 패치 크기가 커진다면 넓은 범위의 이미지를 인식하는데 뛰어남 → 컨텍스트 인식에 탁월 but 지역화에는 한계(넓은 범위를 한 번에 인식하므로)
        - U-Net은 컨텍스트 인식과 지역화 트레이드오프 문제 개선함

> 💡 지역화(localization)
> : 이미지 객체(고양이) 위치 정보를 출력해 주는 것. 주로 바운딩 박스 사용
> 바운딩 박스의 네꼭지점 픽셀 좌표 출력 x 왼쪽 위, 오른쪽 아래 좌표 출력함

- U-Net 구조 - FCN을 기반으로 구축되어 있음
    - 수축 경로(contracting path) : 컨섹스트 포착
    - 확장 경로(expansive path): 특성 맵을 업 샘플링하고 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합하여 정확한 지역화를 수행함
    - 3 × 3 합성곱이 주를 이름. 각 합성곱 블록은 3 x 3 합성곱 두 개로 구성 + 그 사이에 드롭아웃 있음



- 왼쪽 수축 경로에서의 블록 - 3 x 3 합성곱 두 개로 구성된 것이 네 개가 있는 형태. 각 블록은 최대 풀링을 이용하여 크기를 줄이면서 다음 블록으로 넘어감
- 오른쪽 확장 경로 - 합성곱 블록에 up-conv라는 것을 앞에 붙임. 수축 과정에서 줄어든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태

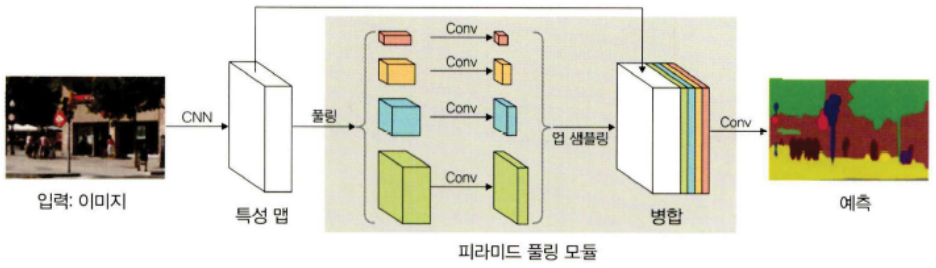
  → 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링과 업 샘플링을 순서대로 반복하는 구조로 되어 있음

6.3.4 PSPNet

- PSPNet(Pyramid Scene Parsing Network): CVPR(The IEEE Conference on Computer Vision and Pattern Recognition) 2017에서 발표된 시멘틱 분할 알고리즘
- 완전연결층의 한계를 극복하기 위해 피라미드 풀링 모듈을 추가
- 훈련 과정
  1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 함
     - 1 × 1, 2 × 2, 3 × 3, 6 × 6 크기로 풀링 수행(1 x 1 크기의 특성 맵은 가장 광범위한 정보를 담음)
     - 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담는다고 이해하자
  2. 이후 1 x 1 합성곱을 사용하여 채널 수 조정
     - 풀링층 개수 N개라고 하면 출력 채널 수 = 입력 채널 수 / N
  3. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링함

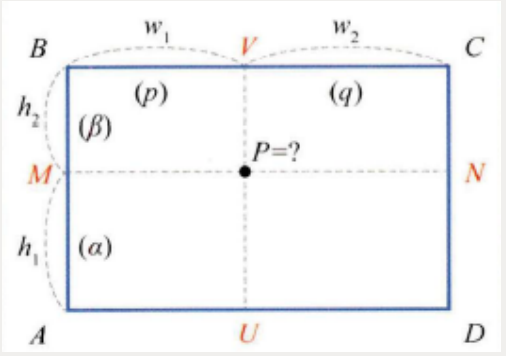
- 이 과정에서 양선형 보간법(bilinear interpolation) 사용됨

4. 원래의 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들을 병합함



양선형 보간법(bilinear interpolation)

- 앞서 보간법이란? 화소 값을 할당받지 못한 영상의 품질을 안 좋을 수밖에 없음. 이때 빈 화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법. 선형과 양선형 보간법이 있음
- 선형 보간법: 원시 영상의 화소 값 두 개를 사용하여 원하는 좌표에서 새로운 화소 값 계산
- 양선형 보간법: 화소당 선형 보간을 세 번 수행하며, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻음
    - 예제 - 직사각형의 네 꼭지점에 값이 주어져 있을 때, 이 사각형 내부에 있는 임의의 점(P)에 대한 값을 추정해보자



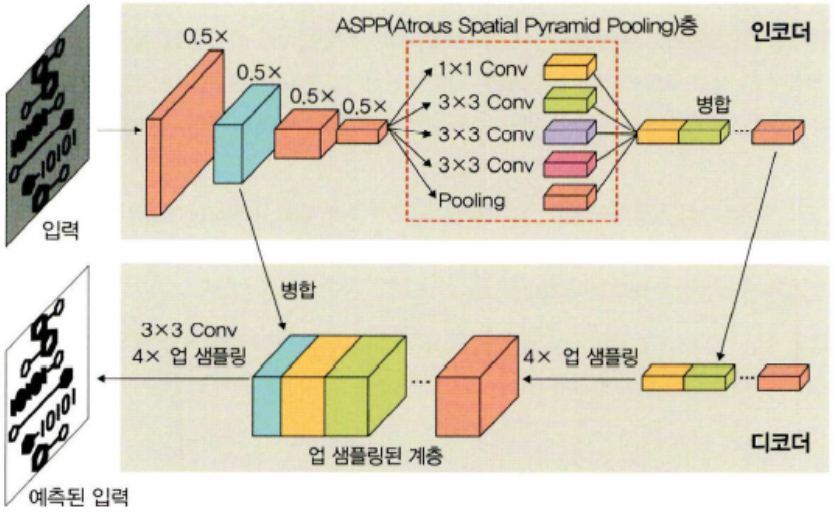
        - 점 P에서 x축 방향으로 거리를 w_1, w_2 , y축 방향으로 거리를 h_1, h_2라고 하자. 이때 알려진 네 점에서 데이터 값을 A, B, C, D 라고 하면
          양선형 보간법에 따라 점 P의 값은 다음과 같이 계산됨

$$P = q(\beta A + \alpha B) + p(\beta D + \alpha C)$$
$$= q\beta A + q\alpha B + p\beta D + p\alpha C$$

6.3.5 Deeplabv3/DeepLabv3+

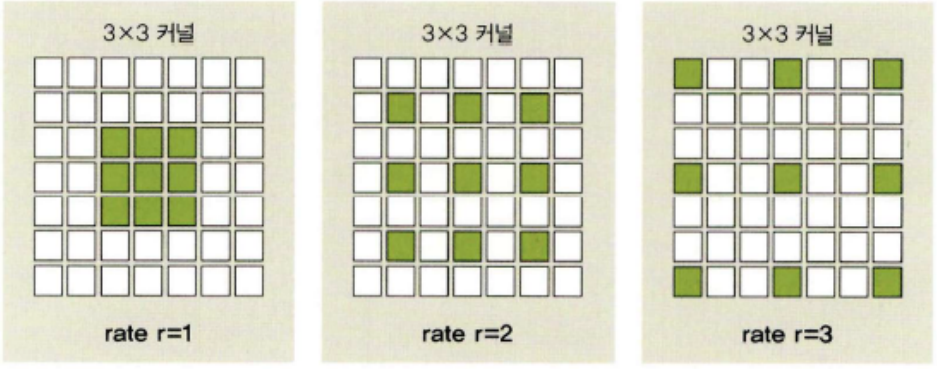
- 완전연결층 단점 보완 위해 Atrous 합성곱 사용하는 네트워크
- 인코더와 디코더 구조를 가짐 - 불가능했던 인코더에서 추출된 특성 맵의 해상도를 Atrous 합성곱을 도입하여 제어할 수 있도록 함

그림 6-54 DeepLab의 인코더-디코더 구조



- Atrous 합성곱

그림 6-55 Atrous 합성곱

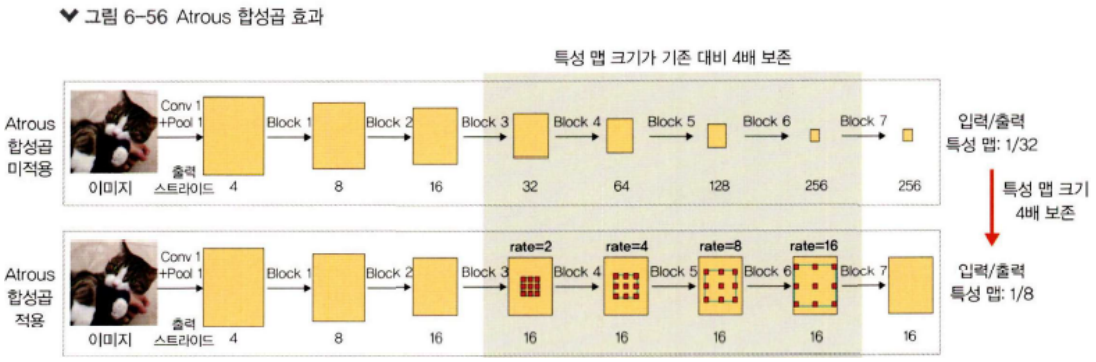


  - 필터 내부에 빈 공간을 둔 채로 작동 → 얼마나 많은 빈 공간을 가질지 결정하는 파라미터 rate

- rate r = 1일 경우 기존 합성곱과 동일하게빈 공간을 가짐, r이 커질수록 빈 공간은 더 많아진다

- 이미지 분할 분야에서 많이 사용
  - 이미지 분할에서는 수용 영역을 확대하여 특성을 찾는 범위를 넓게 해주기 때문에 높은 성능을 위해서는 수용 영역의 크기가 중요
  - Atrous 합성곱 활용하면 파라미터 수 늘리지 않으면서도 수용 영역을 크게 키울 수 O
- 일반적인 CNN 적용 → 출력은 입력의 1/32배가 됨 / Atrous 합성곱 적용 → 1/8로 줄어듬

  → 특성 맵 크기가 기존 대비 4 배 보존





수용 영역
: 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미
마찬가지로 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 연관성은 높지만, 거리가 멀어질수록 그 영향은 감소하게 됨. 영상 전체 영역에 대해 서로 동일한 중요도 부여하여 처리하는 대신, 특정 범위 한정해서 처리하여 효과적으로 훈련을 수행하는 것이 수용 영역